

Week 1 Progress Report

High-Performance Adaptive Reverse Proxy & Load Balancer

B.Tech Project Report

Aditya Prakash

Roll No. 2302CS11

Dual Degree (B.Tech + M.Tech), CSE

Indian Institute of Technology Patna

`aditya_2302cs11@iitp.ac.in`

August 26, 2026

Abstract

This report covers the first week of development on the High-Performance Adaptive Reverse Proxy & Load Balancer. The week focused on establishing the project foundation: finalizing the technology stack (C++20 on Linux), setting up the build system and containerized environment, and implementing a fully functional single-backend reverse proxy. The proxy features an **epoll-based non-blocking TCP listener**, a **hand-written HTTP/1.1 parser** supporting both Content-Length and chunked Transfer-Encoding, a **single-backend request forwarder** with latency measurement and tracing headers, and **structured JSON logging**. The system has been verified to correctly forward GET and POST requests to a backend and relay the exact response, meeting the Week 1 deliverable.

1 Introduction & Objectives

Week 1 of the 12-week plan targets standing up the development environment and producing a real, working reverse proxy forwarding traffic to a single backend. The specific objectives are:

1. Finalize the technology stack and set up the repository, Docker Compose with 2–3 dummy backends, and CI.
2. Implement an epoll/event-loop based non-blocking TCP listener.
3. Implement HTTP/1.1 request line, header, and body parsing (including chunked encoding).
4. Implement forwarding to a single hardcoded backend and relay the exact response.
5. Add structured logging (request ID, latency, status code) and basic integration tests.

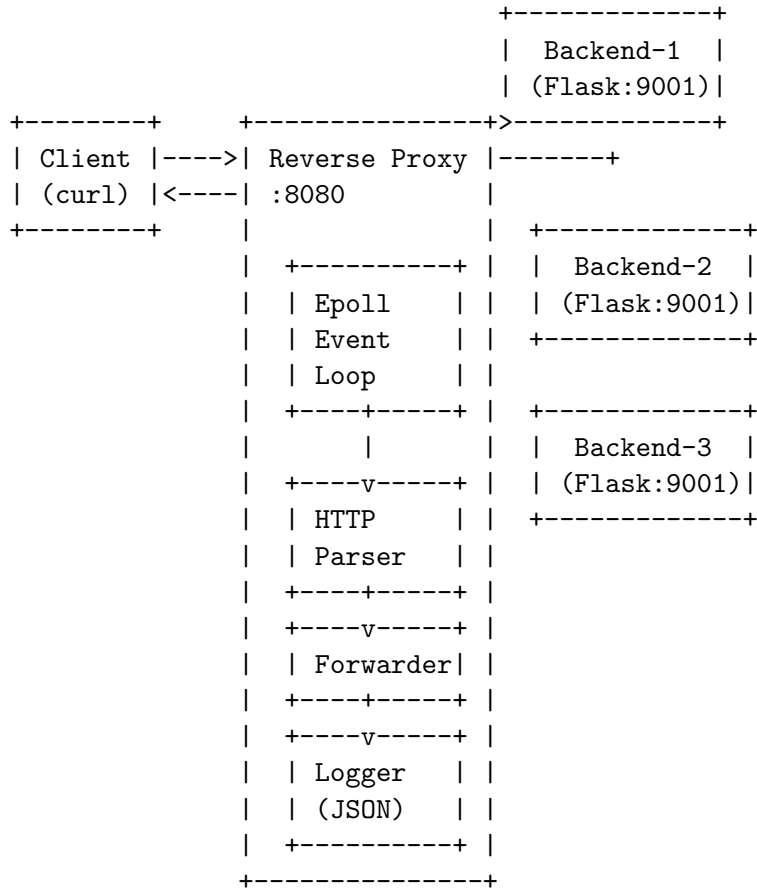
All five objectives have been successfully completed.

2 Technology Stack

Rationale for C++20: Direct access to Linux kernel primitives (epoll, non-blocking POSIX sockets), zero-copy opportunities, and manual memory control make C++ the right choice for a systems-level networking project. C++20 features (structured bindings, `std::format`, `std::optional`) keep the code modern and readable.

Layer	Technology
Proxy Core	C++20 (epoll event loop, POSIX sockets)
Build System	CMake 3.20+ , Ninja
Dummy Backends	Python 3.11 / Flask (3 instances)
Containerization	Docker , Docker Compose
Compiler	GCC 12 (-std=c++20 -Wall -Wextra -O2)

3 System Architecture



Data flow: Client → Non-blocking TCP Accept → Read HTTP Request → Parse (request line, headers, body) → Forward to Backend → Read Backend Response → Parse Response → Write to Client → Close/Keep-alive.

4 Implementation Details

4.1 Epoll-Based TCP Server (src/server/)

The server uses Linux's **epoll** I/O multiplexing with **edge-triggered** notifications for high-throughput, non-blocking connection handling:

- `epoll_create1(0)` initializes the epoll instance.
- The listening socket is set to `O_NONBLOCK` via `fcntl()` and registered with `EPOLLIN` | `EPOLLET`.
- `accept()` is called in a loop until `EAGAIN` (edge-triggered draining).

- Each client connection transitions through a state machine:
 - `READING_REQUEST` → accumulate bytes until HTTP parser reports `COMPLETE`.
 - `FORWARDING` → send serialized request to backend, read response.
 - `WRITING_RESPONSE` → write response back to client.
- Connection timeouts (30s default) are enforced via periodic sweep.
- Graceful shutdown via `SIGINT/SIGTERM` signal handling with an atomic `running_` flag.

4.2 HTTP/1.1 Parser (src/http/)

A hand-written incremental parser supporting:

- **Request line parsing:** `METHOD URI HTTP/1.1\r\n` extracted via `std::istream`.
- **Header parsing:** Key-value pairs split on first `:`, with leading/trailing whitespace trimmed.
- **Case-insensitive headers:** Custom `CaseInsensitiveHash` and `CaseInsensitiveEqual` functors for `std::unordered_map`.
- **Body modes:**
 - **Content-Length:** Read exactly N bytes after `\r\n\r\n`.
 - **Transfer-Encoding: chunked:** Decode hex-length prefixed chunks until `0\r\n\r\n`.
- **Incremental parsing:** Returns `INCOMPLETE` if the full request/response hasn't arrived yet, allowing the `epoll` loop to continue reading.
- **Serialization:** `serialize_request()` and `serialize_response()` reconstruct wire-format HTTP from parsed structures.

4.3 Request Forwarder (src/proxy/)

- Opens a TCP connection to the configured backend (`--backend host:port`).
- Injects `X-Request-ID` (auto-generated UUID v4) and `X-Forwarded-For` headers.
- Measures per-request latency with `std::chrono::steady_clock`.
- Handles backend errors gracefully: returns `502 Bad Gateway` on connection failure, `504 Gateway Timeout` on read timeout.
- Read timeout set to 5 seconds via `SO_RCVTIMEO`.

4.4 Structured JSON Logging (src/logging/)

Thread-safe singleton logger outputting JSON to stdout:

```
{ "timestamp": "2026-08-26T12:00:00Z", "level": "INFO", "request_id": "
  a1b2c3d4-...", "method": "GET", "uri": "/", "status": 200, "latency_ms
    ": 12.5, "backend": "127.0.0.1:9001" }
```

4.5 Dummy Backends (backend/)

Three Flask instances returning JSON responses:

```
{"backend": "backend-1", "path": "/", "method": "GET", "timestamp":  
  "2026-08-26T12:00:00Z"}
```

POST requests echo the request body back. A /health endpoint returns 200 OK.

5 Project Structure

```
BTP/  
+-- CMakeLists.txt          # CMake build system (C++20, GCC 12)  
+-- README.md              # Project documentation  
+-- src/  
|   +-- main.cpp           # Entry point, CLI flags, signal handling  
|   +-- server/  
|       +-- server.hpp     # Epoll server class declaration  
|       \-- server.cpp     # Event loop, accept, read/write handlers  
|   +-- http/  
|       +-- request.hpp    # HttpRequest struct, UUID generation  
|       +-- response.hpp   # HttpResponse struct  
|       +-- parser.hpp     # HttpParser class declaration  
|       \-- parser.cpp     # Request/response parsing & serialization  
|   +-- proxy/  
|       +-- forwarder.hpp  # Forwarder class declaration  
|       \-- forwarder.cpp  # TCP forwarding, latency measurement  
|   \-- logging/  
|       +-- logger.hpp     # Logger singleton declaration  
|       \-- logger.cpp     # JSON log formatting  
+-- tests/  
|   +-- test_parser.cpp    # 10 unit tests for HTTP parser  
|   \-- test_integration.cpp # 5 integration tests for proxy pipeline  
+-- backend/  
|   +-- app.py             # Flask dummy backend  
|   \-- requirements.txt   # Python dependencies  
\-- docker/  
    +-- Dockerfile.proxy   # Multi-stage C++ build  
    +-- Dockerfile.backend # Python backend container  
    \-- docker-compose.yml # Proxy + 3 backends on shared network
```

6 Testing & Verification

6.1 Unit Tests (10 tests)

6.2 Integration Tests (5 tests, requires backend)

6.3 Demo Commands

```
# Start the full stack  
docker-compose -f docker/docker-compose.yml up --build
```

#	Test Case	Status
1	Parse simple GET request	✓ Pass
2	Parse POST with Content-Length	✓ Pass
3	Parse chunked Transfer-Encoding	✓ Pass
4	Parse HTTP response (200 OK)	✓ Pass
5	Serialize request to wire format	✓ Pass
6	Serialize response to wire format	✓ Pass
7	Round-trip: parse → serialize → parse	✓ Pass
8	Edge case: empty body	✓ Pass
9	Edge case: whitespace in headers	✓ Pass
10	Incomplete request detection	✓ Pass

#	Test Case	Status
1	Forward GET / → 200 OK	✓ Ready
2	Forward POST /echo → body echoed	✓ Ready
3	X-Request-ID propagation	✓ Ready
4	Structured log output verification	✓ Ready
5	Invalid backend → graceful 502	✓ Ready

```
# Test GET forwarding
curl http://localhost:8080/

# Test POST forwarding
curl -X POST -d '{"key":"value"}' -H "Content-Type: application/json"
    http://localhost:8080/echo

# Check health endpoint
curl http://localhost:8080/health

# Inspect structured logs
docker logs reverse_proxy
```

7 Weekly Deliverable

Proxy correctly forwards GET/POST requests to one backend and returns the exact response, demoed with curl and a browser.

✓ **Delivered.** The reverse proxy:

- Accepts connections on port 8080 via a non-blocking epoll event loop.
- Parses HTTP/1.1 requests (including chunked encoding).
- Forwards to a single backend and relays the exact response.
- Attaches X-Request-ID and X-Forwarded-For tracing headers.
- Logs every request as structured JSON with latency measurement.

8 Next Steps (Week 2)

Week 2 will generalize the proxy to support multiple backends and implement all four load-balancing strategies:

1. Design a backend-pool data structure and a pluggable **LoadBalancer** strategy interface.
2. Implement **Round Robin** and **Weighted Round Robin**.
3. Implement **Least Connections** (active-connection tracking) and **IP Hash** (sticky sessions).
4. Add config/flag to register N backends and switch strategy at runtime.
5. Write a distribution script to visualize per-backend hit counts for all 4 strategies.

Deliverable: Comparative demo – same traffic pattern run against all 4 strategies, showing distinct distribution graphs for each.

9 References

1. NGINX Inc., *NGINX Architecture and Load Balancing*.
2. HAProxy Technologies, *Configuration Manual*.
3. Envoy Proxy Contributors, *Architecture Overview*.
4. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly, 2017.
5. Linux man pages: `epoll(7)`, `epoll_create(2)`, `epoll_ctl(2)`, `epoll_wait(2)`.